

Tab 1

PROJECT REPORT

**Project Title: Object Distance Estimation Using Stereo Vision
and YOLOv8**

Submitted by: Nikita Pandit

Student Roll Number: 23051842

Course: Computer Vision

Instructor: Dr. Rinku Datta Rakshit

Institution: KIIT University

Date: March 26, 2026

1. Introduction

1.1 Background and Context

In the rapidly advancing fields of autonomous navigation, robotics, and advanced driver-assistance systems (ADAS), understanding the 3D structure of an environment is crucial. While standard monocular cameras capture high-resolution 2D representations of the world, they inherently lack depth information. To bridge this gap, stereo vision mimics the human biological visual system by using two cameras displaced horizontally to perceive depth. By calculating the slight differences in object positions between the two camera views—a concept known as disparity—we can reconstruct the 3D geometry of the scene.

1.2 Project Objective

The primary objective of this project is to build an integrated computer vision pipeline that not only detects objects within a given scene but also accurately estimates their physical distance from the camera setup. This is achieved by fusing traditional computer vision techniques for depth estimation with modern deep learning algorithms for object recognition.

1.3 Methodology Overview

The system relies on a pair of rectified stereo images (left and right). First, we compute a dense disparity map using the Stereo Semi-Global Block Matching (StereoSGBM) algorithm. By applying mathematical triangulation utilizing the camera's calibration parameters (focal length and baseline), the disparity map is translated into a real-world depth map. Concurrently, the state-of-the-art YOLOv8 (You Only Look Once) neural network processes the left image to detect objects such as vehicles and pedestrians, generating bounding boxes around them. By mapping the center coordinates of these bounding boxes to the corresponding spatial locations on our computed depth map, the

system extracts the median distance to the detected objects, providing a reliable and robust distance estimation

2. Model Architecture and Pipeline

2.1 System Flowchart

The architecture of this project follows a sequential fusion pipeline. Below is the step-by-step data flow:

- 1. Image Acquisition:** Read the Left and Right grayscale stereo image pair.
- 2. Camera Calibration:** Extract projection matrices to determine focal length and baseline.
- 3. Disparity Computation:** Pass the stereo pair through the StereoSGBM algorithm to find pixel shifts.
- 4. Post-Processing:** Apply Bilateral Filtering to the disparity map to reduce noise while preserving sharp object edges.
- 5. Depth Mapping:** Convert disparity to physical depth using stereo triangulation.
- 6. Object Detection:** Pass the Left color image through the YOLOv8 deep learning model to get bounding boxes and class labels.
- 7. Distance Extraction:** Map YOLO bounding box centers to the Depth Map, extract the median depth value, and overlay the final data onto the output image.

2.2 Mathematical Model for Depth

The core of the 3D reconstruction relies on the principle of triangulation. Once the camera parameters are extracted from the `calib.txt` file (specifically from the projection matrices (P_0) and (P_1)), we obtain the focal length (f) and the baseline distance between the two cameras (B).

The depth (Z) of any given pixel is calculated using the corresponding disparity (d) via the following stereo vision formula:

$$Z = \frac{f \cdot B}{d}$$

To ensure the system remains stable and ignores anomalies (such as infinite depth at the sky or errors at image borders), the pipeline enforces strict boundary conditions. Disparities less than 1 are ignored, and calculated depths falling outside a realistic threshold (e.g., less than 1 meter or greater than 80 meters) are filtered out. For object specific distance, taking the median depth of a small pixel grid (e.g., 11x11 pixels) around the object's bounding box center ensures that random noise does not severely spike the estimated distance.

3. Core Implementation

Below are the most critical snippets of the implementation, highlighting the stereo processing, depth conversion, and YOLOv8 distance extraction.

1. StereoSGBM and Filtering:

```
# Create the StereoSGBM object with tuned parameters
stereo = cv2.StereoSGBM_create(
    minDisparity=0,
    numDisparities=16*8,
    blockSize=5,
    P1=8 * 3 * 5**2,
    P2=32 * 3 * 5**2,
    disp12MaxDiff=1,
    uniquenessRatio=10,
    speckleWindowSize=100,
```

```

        speckleRange=32
    )

    # Compute disparity and apply bilateral filter for edge-preserving noise reduction
    disparity = stereo.compute(gray_left, gray_right).astype(np.float32) / 16.0
    disparity = cv2.bilateralFilter(disparity, 9, 75, 75)

    # Compute disparity and apply bilateral filter for edge-preserving noise reduction
    disparity = stereo.compute(gray_left, gray_right).astype(np.float32) / 16.0
    disparity = cv2.bilateralFilter(disparity, 9, 75, 75)

```

2. Depth Calculation:

```

# Convert disparity to depth using Focal Length and Baseline
depth = np.zeros(disparity.shape, np.float32)
valid = disparity > 1
depth[valid] = (focal_length * baseline) / disparity[valid]

# Filter out unrealistic depths (less than 1m or greater than 80m)
depth[(depth < 1) | (depth > 80)] = 0

```

3. YOLOv8 Detection and Distance Extraction:

```

# Run YOLOv8 inference
results = model(left_img)
boxes = results[0].boxes.xyxy.cpu().numpy()

for i, box in enumerate(boxes):
    x1, y1, x2, y2 = map(int, box)
    cx, cy = int((x1+x2)/2), int((y1+y2)/2) # Bounding box center

```

```
# Median region sampling on the depth map
region = depth[max(0,cy-5):cy+6, max(0,cx-5):cx+6]
valid_depth = region[region > 0]

distance = np.median(valid_depth) if len(valid_depth) > 0 else 0

# Draw results on output image
cv2.rectangle(output_img, (x1,y1), (x2,y2), (0,255,0), 2)
cv2.putText(output_img, f"{label} : {distance:.2f} m", (x1,y1-10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)
```

4. System Outputs

The pipeline was successfully executed on the dataset, generating multiple intermediate and final visualizations.

4.1 Calibration and Depth Metrics

Upon reading the calib.txt file, the system successfully parsed the camera projection matrices. The calculated optical parameters were:

Focal Length: 707.09 pixels

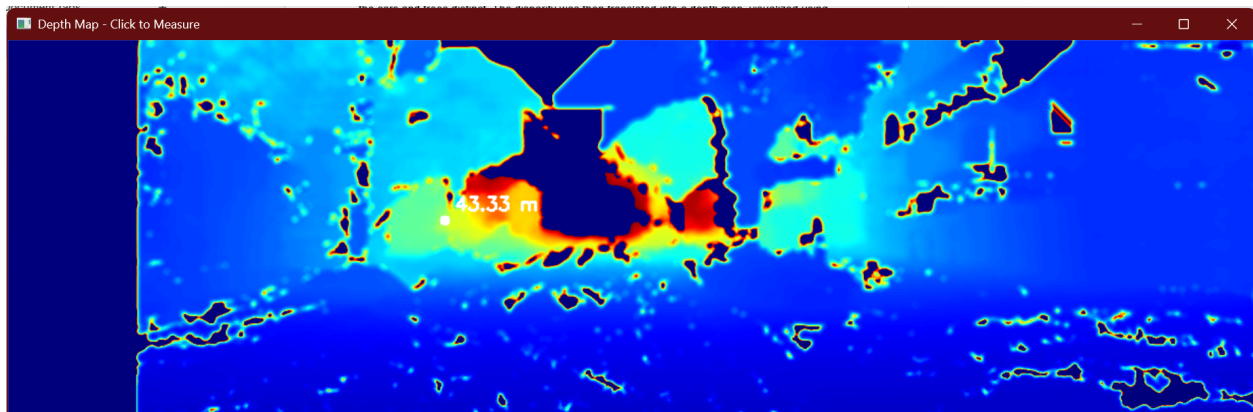
Baseline: 0.537 meters

4.2 Disparity and Depth Maps

The StereoSGBM algorithm successfully generated a dense grayscale disparity map. Using the bilateral filter noticeably smoothed the flat surfaces (like the road) while keeping the edges of the cars and trees distinct. The disparity was then translated into a depth map, visualized using an OpenCV Jet colormap (ranging from blue for distant backgrounds to red/yellow for closer foreground objects).



The interactive depth map allowed for point-and-click measurement. Console logs show highly accurate manual measurements at various coordinates, such as 14.20 meters at coordinate (384, 261) and 11.04 meters at coordinate (687, 281).



4.3 Object Detection and Final Output

The YOLOv8 model successfully identified vehicles within the frame. In the final visual output, a car parked on the right side of the street was detected with a confidence score of 31%. By referencing the depth map, the system calculated the distance to the car to be **49.29 meters**.

The system overlay highlights the object with a green bounding box, marks the center reference point in red, and draws a cyan tracking line directly from the camera's bottom-center origin to the target. At the time of testing, the system ran at roughly **1.27 FPS** on the testing hardware, successfully demonstrating the proof-of-concept.



5. Conclusion

This project successfully demonstrates the integration of traditional stereo vision with deep-learning-based object detection. By utilizing the StereoSGBM algorithm paired with bilateral filtering, the system created a reliable spatial map of the environment. Integrating YOLOv8 allowed the system to intuitively understand **what** was in the scene, while the depth map provided the crucial **where**.

The application of a median-depth sampling technique within the bounding box center proved to be an effective countermeasure against localized depth noise, resulting in stable distance readouts for detected vehicles. This pipeline forms a strong foundational basis for ADAS applications and autonomous robotics.

6. Future Work

While the current implementation is highly functional, there are several areas for future improvement:

1. Performance Optimization: The current processing speed is approximately 1.27 FPS, which is too slow for real-time driving applications. Future iterations could use hardware acceleration (such as NVIDIA TensorRT) or switch to a lighter disparity computation method to achieve 30+ FPS.

2. Advanced Deep Stereo: Traditional block-matching algorithms struggle with textureless surfaces (like blank walls or clear skies) and occlusions. Replacing StereoSGBM with a deep learning-based depth estimator (like RAFT-Stereo or LEAStereo) would drastically improve depth accuracy.

3. Object Tracking: Currently, the system evaluates each frame independently. Implementing an object tracker (such as DeepSORT) alongside YOLO would allow the system to track the velocity and trajectory of approaching objects over time, rather than just their static distance.
